

iOS Interview Handbook

Interview Preparation Roadmap

A focused, intensive plan for mid-to-senior iOS developers preparing for interviews. This roadmap maps directly to the chapters in this edition and assumes **4 to 5 hours of daily study**, with weekends reserved for problem-solving, catch-up, and revision.

The next few years will not be kind to developers who coast on past experience. Drop the excuses, commit to your preparation roadmap, and outwork the version of you that has been postponing this.

How to Use This Roadmap

This roadmap is designed for a 45-day intensive preparation with approximately 4 to 5 hours of daily study. The schedule balances core iOS topics with two parallel tracks (Mobile System Design and Problem Solving / DSA) so you are not cramming them at the end.

Time Allocation: Monday to Friday (~4 hours/day):

- 2 hours on Core topic of the day: Read the chapter, write code in playgrounds, and take notes.
- 1 hour on Mobile System Design (Monday, Wednesday, Friday): Pick one design problem at the start of the week and progress through it across the week. By the end of the week, you should have covered the entire problem including HLD and LLD.
- 1 hour on Problem Solving / DSA (Tuesday, Thursday): Solve 1 to 2 problems in Swift. Focus on the data structure or pattern assigned for that week.

Time Allocation: Saturday & Sunday (~5 to 6 hours/day):

- 1 to 2 hours on Problem Solving / DSA (Saturday): Solve 3 to 4 problems, including at least one medium problem to stretch your thinking.

- 2 to 3 hours on catch-up: Cover any topic that was skipped, rushed, or felt shaky during the week. This is also where mock interviews are scheduled.
- Remaining hours on weekly revision: Go through your notes, flashcards, and "gotchas" log from the week. Revision is non-negotiable. It is what makes the knowledge stick.

Note: You can adjust the time division of the day according to your availability. Life happens, and what matters is consistency across the 45 days rather than rigid hour blocks.

Practical Approach While Preparing

Reading alone will not get you through interviews. These habits make the difference between knowing a topic and being able to explain it under pressure:

- **If an answer does not click in one reading, read it again.** No worries because some topics (like actors, Sendable, Combine backpressure) take 2 to 3 reads. After the second read, open Swift Playgrounds and try the example yourself.
- **Run every code snippet in Playgrounds.** Do not just trust the output written in the book. Modify the example, break it, and see what happens. That is where real understanding comes from.
- **Maintain a "gotchas" document.** Whenever you get something wrong or are surprised by Swift behavior, log it. Re-read it weekly. By the end of this roadmap, this becomes your personal cheat sheet.
- **Explain out loud.** If you cannot teach a concept in 2 minutes without notes, you do not know it well enough. Record yourself if no one is around.
- **Connect topics to real bugs you have fixed.** When revising memory management, recall an actual retain cycle you debugged. These stories become your best interview answers.
- **Track time-to-answer.** In real interviews, you have around 2 minutes per question. Practice answering within that window. Long-winded answers hurt as much as short ones.
- **Do not skip mocks.** They expose blind spots no amount of reading will.
- **Pace yourself.** Burning out by Day 30 is a common failure mode. Sleep matters more than an extra hour of study.

Mock Interview Strategy

In addition to the self/peer mocks already scheduled in this roadmap, book one mock interview with an expert every two weeks. An expert mock gives you:

- Honest feedback on your communication, not just correctness
- Exposure to the depth of follow-up questions real interviewers ask
- A realistic gauge of where you stand for your target level

Peer mocks help with practice. Expert mocks help with calibration. You need both.

Phase 0: Self-Assessment (Day 0 - Before You Start)

Before Day 1, spend some hours on honest self-assessment. Go through the chapter list and rate yourself 1 (weak) to 5 (strong) on each. Common weak areas for mid-senior iOS developers include:

- Swift Concurrency (async/await, actors, structured concurrency, Sendable, MainActor)
- Combine Framework (especially backpressure, custom operators, error handling, debouncing)
- Advanced SwiftUI (identity, render optimization, @Observable, NavigationStack, State Management)
- Memory Management deep dive (ARC edge cases, retain cycle debugging, weak vs unowned)
- Mobile System Design (offline-first, sync engines, scalability, caching strategies, pagination)
- SOLID Principles & Architecture (Clean Architecture, Repository pattern, MVVM-C)
- Core Data internals (faulting, merge policies, NSFetchedResultsController, migrations)
- Generics & Type Erasure
- Protocol-Oriented Programming (protocol composition, conditional conformance)

- Unit Testing legacy or tightly-coupled code (mocking, dependency injection for tests)
- Networking depth (background tasks, resumable downloads, token refresh)
- App Performance profiling (Instruments, Time Profiler, Allocations, Leaks)
- DSA in Swift (if you have not practiced in a while)

Write down your weak areas. You will allocate **extra revision time** to these in the buffer days.

Week 1: Swift Foundations & OOP

System Design focus this week: News Feed Design

DSA focus this week: Arrays, Strings, Two Pointers, Sliding Window

The goal of this week is to lock down language fundamentals. Interviewers test these early to gauge depth.

Day 1 - Class, Structure, Actors & Enumeration: Value vs reference types, COW, deep vs shallow copy, recursive enums, actors basics. **System Design:** News Feed requirements and scope.

Day 2 - Properties & Initializers: Stored vs computed, lazy, property wrappers, designated/convenience/failable/required initializers. **DSA:** Array fundamentals (in-place modification, prefix sums, basic patterns).

Day 3 - Functions, Methods & Closures: Higher-order functions, escaping/non-escaping, capture lists, autoclosures, retain cycles in closures. **System Design:** News Feed HLD (feed service, ranking, pagination).

Day 4 - Protocol & Delegation: Protocol extensions, composition, associated types, method dispatch in protocols, PATs vs generics. **DSA:** String fundamentals (character manipulation, palindrome patterns).

Day 5 - Generics & Error Handling: Type constraints, type erasure, where clauses, custom errors, throws/rethrows, async error handling. **System Design:** News Feed LLD (data models, caching layer, infinite scroll).

Day 6 - DSA + Catch-up: Problem solving on mixing Arrays and Strings. Catch up on any rushed topic from this week.

Day 7 - Weekly Revision + Mock Interview #1: Complete revision of this week. Self or peer mock interview.

Week 2: Memory & Concurrency

System Design focus this week: Chat/Messaging App with Offline Support

DSA focus this week: Two Pointers & Sliding Window

This week is the **most critical** for senior-level interviews. Concurrency and memory questions almost always appear.

Day 8 - Memory Management Part 1: Retain cycles, strong references in closures, weak vs unowned, thread safety basics, ARC fundamentals. **System Design:** Chat App requirements and offline-first scope.

Day 9 - Memory Management Part 2: deinit, stack vs heap, multimedia memory optimization, third-party library impact, singleton memory pitfalls, large dataset optimization. **DSA:** Two Pointers patterns (opposite ends, fast/slow, pair sum).

Day 10 - Concurrency Part 1, Traditional: DispatchQueue.main.async vs sync, sync vs async, serial vs concurrent queues, GCD queue types, DispatchGroup, common pitfalls. **System Design:** Chat App HLD (message delivery, sockets, presence).

Day 11 - Concurrency Part 2, Modern: async/await vs GCD, task cancellation, error handling in async, structured concurrency, Swift actors. **DSA:** Sliding Window patterns (fixed window, variable window, longest substring problems).

Day 12 - Concurrency Part 3, Advanced: DispatchSemaphore (wait/signal, use cases), RunLoop, RunLoop vs DispatchQueue, static vs dynamic dispatch, dispatch group vs semaphore. **System Design:** Chat App LLD (local DB schema, message sync, conflict resolution).

Day 13 - DSA + Catch-up: Problem solving on mixing Two Pointers and Sliding Window. Revisit any tricky Array/String problems from the previous week. Catch up on concurrency topics that felt shaky.

Day 14 - Weekly Revision + Mock Interview #2 (Expert): Complete revision of this week. Book an expert mock interview focused on concurrency and memory management.

Week 3: Networking, Combine & UIViewController

System Design focus this week: Photo Sharing App (Feed & Uploads)

DSA focus this week: LinkedList

Day 15 - Networking: URLSession configurations, error handling, background tasks, caching, auth, token refresh, resumable downloads, concurrent requests.

System Design: Photo Sharing requirements and scope.

Day 16 - Combine Framework Part 1: Publishers and subscribers, operators, backpressure, subjects, Combine vs RxSwift, error handling in pipelines. **DSA:** LinkedList basics (traversal, reversal, fast/slow pointers).

Day 17 - Combine Framework Part 2: SwiftUI integration, sink, AnyCancellable, assign operator, real-world use cases, debounce and throttle. **System Design:** Photo Sharing HLD (upload pipeline, CDN, image processing).

Day 18 - UIViewController Lifecycle: viewWillAppear vs viewDidLoad, layout methods, viewWillTransition, loadView, memory warnings, child VC lifecycle, modal presentation lifecycle. **DSA:** LinkedList intermediate (cycle detection, merging lists, removing nodes).

Day 19 - UIKit Part 1: UIView vs CALayer, masksToBounds vs clipsToBounds, frame/bounds/center/transform, Core Animation, Auto Layout conflicts, responder chain, delegate/data source, IBOutlet weak. **System Design:** Photo Sharing LLD (image caching, prefetching, upload queue).

Day 20 - DSA + Catch-up: Problem solving on LinkedList (mix of easy and medium). Catch up on UIKit or Combine if anything felt shaky.

Day 21 - Weekly Revision + Mock Interview #3: Complete revision of this week. Self or peer mock on networking and Combine.

Week 4: UIKit (Continue) & SwiftUI

System Design focus this week: Push Notification & Deep Linking Architecture

DSA focus this week: Stack & Queue

Day 22 - UIKit Part 2: Dynamic cell heights, UIWindow, UINavigationController, reusable UI components, prefetching, prepareForReuse, dynamic fonts, app states, main thread UI, IBInspectable/IBDesignable. **System Design:** Push Notification requirements (silent push, rich notifications, providers).

Day 23 - SwiftUI Basics Part 1: @State, @Binding, @ObservedObject, @EnvironmentObject, declarative syntax, @Environment, NavigationLink/View, ViewModifiers, performance, ObservableObject, GeometryReader, @AppStorage/@SceneStorage, SwiftUI vs UIKit. **DSA:** Stack patterns (parentheses matching, monotonic stack, expression evaluation).

Day 24 - SwiftUI Basics Part 2: Containers, struct for views, .task() vs .onAppear(), using UIKit in SwiftUI, ObservableObject change announcement, localization, zIndex, ViewBuilders. **System Design:** Push Notification HLD (APNs flow, payload design, server architecture).

Day 25 - Advanced SwiftUI Part 1: View identity and .id(), View struct immutability, body evaluation, EquatableView, @Observable vs ObservableObject, @StateObject vs @ObservedObject, global state, GeometryReader vs Layout protocol, NavigationStack and NavigationPath, task cancellation, massive ViewModels, Dependency Injection, mocking previews. **DSA:** Queue patterns (BFS prep, circular queue, deque, queue with stacks).

Day 26 - Advanced SwiftUI Part 2: Optimizing large lists, @MainActor, UIViewControllerRepresentable pitfalls, ScrollViewReader, Swift 6 strict concurrency, Sendable, offloading heavy work, profiling body evaluations, Xcode profiling tools, closures in views, side effects, decoupled data flow, Repository pattern, testability. **System Design:** Deep Linking architecture (universal links, custom schemes, routing).

Day 27 - DSA + Catch-up: Problem solving mixing Stack and Queue. Revisit tricky LinkedList problems from the previous week. Catch up on SwiftUI topics if needed.

Day 28 - Weekly Revision + Mock Interview #4 (Expert): Complete revision of this week. Book an expert mock interview focused on UI frameworks and SwiftUI architecture.

Week 5: Architecture, SOLID & Performance

System Design focus this week: Image Caching System (like SDWebImage)

DSA focus this week: Trees & Recursion

Day 29 - SOLID Principles Part 1: What SOLID is and why it matters, refactoring legacy code, Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation. **System Design:** Image Caching requirements (memory cache, disk cache, eviction).

Day 30 - SOLID Principles Part 2: Dependency Inversion, convincing teams to adopt SOLID, DI vs DIP, code reusability and modularity, drawbacks of strict adherence. **DSA:** Recursion fundamentals (base cases, recursion tree, classic problems).

Day 31 - Architectures & Design Patterns Part 1: Design patterns importance, Builder, Observer (NotificationCenter), Delegate, Factory Method, Adapter, Facade, choosing the right pattern, Mediator, VC role, modular architecture, DI. **System Design:** Image Caching HLD (cache hierarchy, async loading, cancellation).

Day 32 - Architectures & Design Patterns Part 2: SwiftUI impact on architecture, separation of concerns, decoupling, MVC components and pitfalls, MVVM popularity, ViewModel role, Clean Architecture, dependency handling, feature flags, conditional compilation. **DSA:** Binary Tree basics (traversals: inorder, preorder, postorder, level order).

Day 33 - App Performance: Caching mechanisms, image loading, background processing, battery optimization, smooth scrolling, memory leaks, app startup time, Instruments and profilers, large dataset rendering, lazy loading, app size reduction, third-party library impact. **System Design:** Image Caching LLD (LRU cache, thread-safe access, disk persistence).

Day 34 - DSA + Catch-up: Problem solving on Recursion and Binary Trees. Catch up on architecture topics.

Day 35 - Weekly Revision + Mock Interview #5: Complete revision of this week. Self or peer mock focused on architecture and system design.

Week 6: Security, Persistence & Final Polish

System Design focus this week: Offline-First Sync Engine

DSA focus this week: BFS & DFS

Day 36 - App Security Part 1: Keychain Services, HTTPS, App Transport Security (ATS), SSL/TLS. **System Design:** Offline-First Sync requirements (conflict resolution, sync triggers).

Day 37 - App Security Part 2: Symmetric vs asymmetric encryption, secure user sessions, logging and monitoring for security incidents. **DSA:** BFS patterns (level-order traversal, shortest path in unweighted graph, grid problems).

Day 38 - Data Persistence Part 1: Choosing storage methods, memory vs disk caching, UserDefaults limits, UserDefaults migrations, Core Data stack, NSManagedObject vs NSManagedObjectContext, main vs background context, faulting, lightweight vs heavyweight migrations, unit tests for Core Data, merge policies. **System Design:** Offline-First Sync HLD (queue design, delta sync, server reconciliation).

Day 39 - Data Persistence Part 2: Delete rules, preventing main thread blocking, preventing Core Data bloat, SwiftData vs Core Data, Core Data to SwiftData migration, Core Data encryption, N+1 problem, undo/redo, denormalization for performance, NSFetchedResultsController. **DSA:** DFS patterns (tree DFS, backtracking introduction, path problems).

Day 40 - Unit Testing, TDD, UI Testing & Swift Testing: XCTest vs Swift Testing, AAA pattern, async testing, testing legacy code, code coverage, testing private methods, TDD three laws and Red-Green-Refactor, TDD pros/cons, TDD for networking, TDD for shopping cart, unit vs UI testing, permissions testing, race

conditions, structuring tests for complex dependencies. **System Design:** Offline-First Sync LLD (operation queue, retry logic, error states).

Day 41 - DSA + Machine Round Practice: Problems solving on BFS and DFS. Pick one **Machine Round assignment** (e.g. build a small app with networking + persistence + tests in 3 hours).

Day 42 - Weekly Revision + Mock Interview #6 (Expert): Complete revision of this week. **Book an expert mock interview:** full loop simulation (coding + system design + iOS deep dive).

Week 7: Miscellaneous, Live Coding & Game Day

Day 43 - Miscellaneous Part 1 + Live Coding: Extensions limitations, type aliases, immutable collections, append/insert, for-in vs for-each, Codable customization, date formatting, KVO, AppDelegate vs SceneDelegate, Equatable vs Comparable, final keyword, singletons, optionals. **Live Coding practice:** build a paginated list with cache in 60 minutes.

Day 44 - Miscellaneous Part 2 + Behavioural Prep: Optional type, Any/AnyObject/Generic, == vs ===, singletons, self vs Self, @discardableResult, access control levels, switch vs if-else, leading vs left constraint, reuseIdentifier, Hashable inherits Equatable, static vs dynamic libraries, Array vs NSArray, optional binding vs chaining, Float/Double/CGFloat, immutability. Prepare 4 to 5 STAR-format behavioural stories.

Day 45 - Game Day Mode: Just relax now. Prepare your "gotchas" document if pending. **Important:** Again, make a list of the topics that you feel not fully confident about.

Take a 2-days break

Beyond Day 45: Continuing the Journey

The 45 days are not the end. Interview preparation is an ongoing process, and you should plan to continue working on the following areas after Day 45:

Continue DSA & Problem Solving

Pick up where the roadmap left off and continue with the remaining topic groups, 2 weeks each:

- **HashMap, HashSet & Heap (2 weeks):** Frequency maps, set operations, top-K problems, priority queue patterns, heap-based scheduling.
- **Dynamic Programming Basics & Greedy (2 weeks):** 1D and 2D DP, memoization vs tabulation, classic DP problems (knapsack, LIS, edit distance), greedy choice property, interval problems.
- **Mixed problem sets:** After completing all topic groups, switch to mixed daily problems to simulate real interview conditions where you do not know which pattern to apply.

Continue Mobile System Design

Expand beyond the six designs covered during the 45 days. Suggested next topics:

- Video Streaming App (HLS, adaptive bitrate, prefetching)
- Ride-Sharing Client (real-time location, driver matching)
- File Downloader with Resumable Downloads
- Analytics SDK Design (batching, retry, privacy)
- Authentication & Token Refresh Flow
- Real-Time Collaboration App (operational transforms, CRDT basics)
- Maps & Location-Based App (tile caching, geofencing)
- E-commerce Cart & Checkout System

Cover one design per week with the same HLD-then-LLD approach used during the roadmap.

Continue on Weak Areas

By Day 45, you will have a clear sense of which topics still feel shaky. Common ones at this stage:

- Swift 6 strict concurrency and Sendable in real codebases
- Combine custom operators and backpressure handling
- Clean Architecture in a multi-module project
- Core Data + SwiftData hybrid scenarios
- Advanced SwiftUI rendering optimization

Dedicate 1 to 2 weeks per weak area for deep practice. Build small focused projects that force you to use these concepts.

Key Principles to Follow

- **Do not just read, write code.** Every concept should be backed by a small Xcode playground.
 - **Maintain a "gotchas" document.** Whenever you get something wrong, log it. Re-read it weekly.
 - **Explain out loud.** If you cannot teach a concept in 2 minutes, you do not know it well enough.
 - **Do not skip mocks.** They expose blind spots no amount of reading will.
 - **Sleep matters.** Burning out by Day 15/30 is a common failure mode. Pace yourself.
-

Suggested Side Projects (Optional, If You Have Buffer Time)

These projects are **not part of the 45-day roadmap**. Build them only if you finish topics early or want hands-on practice during weekends. Each project is chosen to reinforce specific interview-relevant skills.

1. SwiftUI - Offline-First Notes App with Sync: Build a notes app using SwiftUI + SwiftData (or Core Data) with offline-first behavior and background sync to a mock server. Reinforces: SwiftData/Core Data, @Observable, async/await, conflict resolution, Repository pattern, MVVM architecture, unit testing.

2. SwiftUI - Crypto/Stocks Tracker with Live Updates: Build a real-time price tracker using SwiftUI + Combine + URLSession with WebSocket or polling. Include charts, watchlists, and search. Reinforces: Combine pipelines, async streams, NavigationStack, performance optimization for frequent updates, dependency injection, @MainActor usage.

3. UIKit - Image Feed App with Custom Caching: Build an Instagram-style image feed using UIKit + URLSession with your own image caching layer (memory + disk, LRU eviction). Add infinite scroll and prefetching. Reinforces: UICollectionView, prefetching, prepareForReuse, NSCache, GCD, memory management, Auto Layout, custom Flowable layouts.

4. UIKit - Modular Chat UI with Diffable Data Source: Build a chat interface using UIKit with UICollectionViewDiffableDataSource, compositional layout, and a clean separation between message types (text, image, system). Reinforces: modern UIKit (diffable data sources, compositional layout), responder chain, custom cells, MVVM-C or Coordinator pattern, accessibility, dynamic type.

----- *end of roadmap* -----